

FlickIt: Requisitos de la heurística y serendipia.

A partir de las funcionalidades vistas antes vamos a definir los requisitos necesarios para su implementación correcta. Como la mayoría se llevan a cabo de forma ajena al usuario (es decir, sin su conocimiento o “input”), no necesitaran requerimientos de usuario.

HEURÍSTICA

1. Personalización :

- **Requerimientos funcionales:** A la hora de crear un perfil de gustos y exclusiones de un usuario debemos hacer que nuestro programa implemente alguna manera de distinguir que productos se relacionan entre sí (Ya sea de manera positiva o negativa en la búsqueda).
- **Requerimientos no funcionales:** Podemos lograrlo de dos maneras.
 - Crear atributos “ocultos” a los objetos que los relacionen con otros como por ejemplo: Una silla tendría el atributo “(Mueble)” y si el usuario no muestra interés por sillas podemos crear un algoritmo que asuma que tampoco querrá productos con el atributo “(Mueble)”.
 - Otra opción sería, implementar una inteligencia artificial que analice los gustos del usuario e intente generar que producto debería recomendarse.
- **Requerimientos de usuario:** El usuario sera el responsable de expandir su lista de gustos o exclusiones lo cual ya es un elementos clave de nuestro diseño (“swipe left or right”).

2. Participación activa :

- **Requerimientos funcionales:** Ademas del perfil de gustos antes vistos, debemos tener en cuenta el cambio en gustos con el tiempo del usuario. Por esto nuestro programa con el tiempo debe volver a recomendar productos para averiguar si ahora son de interés y dejar de recomendar otros productos cuando el usuario deja de comprarlos/pedirlos.
- **Requerimientos no funcionales:**
 - Con la primera de las propuestas simplemente deberíamos hacer que el “gusto” o “rechazo” a un producto se reduzca con el tiempo volviendo a un valor base para volver a ser recomendado. La IA podría utilizar un concepto similar, aunque también podríamos hacer un reset de sus datos de usuario después de un tiempo.
- **Requerimientos de usuario:** El usuario esta exento de participar en esta funcionalidad.

3. Competitividad :

3.1 Reseñas y fiabilidad:

- **Requerimientos funcionales:** Para poder filtrar productos de esta manera tendremos que añadir a nuestro algoritmo de recomendación una forma de interpretar la fiabilidad del proveedor. Además, debe tener reseñas lo cual pide un sistema de reseñas de productos.
- **Requerimientos no funcionales:**
 - Se pueden alcanzar posiblemente con datos externos a la aplicación (bases de datos o paginas de terceros como Trustpilot) o establecer valores como “compañía”/“vendedor independiente”/ nuestro propio sistema de valoración de productos.
- **Requerimientos de usuario:** En el caso de que sea necesario los usuarios podrán poner reseñas a sus productos comprados para alimentar a nuestro algoritmo de recomendaciones y valoraciones.

3.2 Proximidad geográfica :

- **Requerimientos funcionales:** Rebajar el transporte y tarifas permitiría rebajar el precio de ciertos objetos creando precios más competitivos y incitando más compras. Solo sería necesario la ubicación del usuario (deberá proporcionárnosla o este parámetro estará desactivado).
- **Requerimientos no funcionales:**

- Si el usuario permite usar su ubicación, necesitamos que el algoritmo tenga en cuenta la cercanía de los proveedores en su búsqueda del mejor producto. Podemos hacerlo asignando a los proveedores atributos de ciudad y país.
- **Requerimientos de usuario:** El usuario solo nos tiene que permitir usar su ubicación para que esta funcionalidad se aplique.

3.2 Precio:

- **Requerimientos funcionales:** Otro atributo que añadir a la búsqueda, queremos seleccionar los precios más competitivos (baratos) para aumentar ventas.
- **Requerimientos no funcionales:**
 - El algoritmo seleccionara el producto más barato y de un proveedor fiable posible. Ya sea con IA o con atributos como discutido anteriormente añadirlo a estos no debería ser más que un atributo más que tener en cuenta.
- **Requerimientos de usuario:** El usuario esta exento de participar en esta funcionalidad. (A lo mejor permitir que el usuario elija que gama de productos/ precio quiere buscar).

Serendipia

1. Equilibrio con personalización:

- **Requerimientos funcionales:** Añadir este factor “aleatorio” al algoritmos de búsqueda que permitirá que el usuario descubra nuevos productos sin siquiera tener que buscar activamente.
- **Requerimientos no funcionales:**
 - Para lograr esto podemos crear un algoritmo de búsqueda “aleatoria” que intente seleccionar un producto valido (diferente de los de la lista de gustos y exclusiones) que el usuario no haya visto previamente.
- **Requerimientos de usuario:** El usuario esta exento de participar en esta funcionalidad activamente.

2. Mayor alcance:

- **Requerimientos funcionales:** Intentar variar entre pequeños y grandes negocios en nuestro algoritmo para diversificar nuestras recomendaciones puede causar una “sorpresa positiva” en el usuario. Por ejemplo, a lo mejor pueden interesarle productos artesanales que no suelen ofrecer grandes empresas.
- **Requerimientos no funcionales:**
 - Añadir parámetros que comprueben que haya un balance entre proveedores grandes (por ejemplo Amazon) y pequeños vendedores (otros usuarios o pequeños negocios).
- **Requerimientos de usuario:** El usuario esta exento de participar en esta funcionalidad activamente.